

Processor Design Final Project Report: Physical Memory Protection Integration in the VeeR EH2 Processor

Duru Degimli & Rabia Göksel Demiray
Master in Innovation and Research in Informatics
Universitat Politècnica de Catalunya

June 23, 2026

1 Introduction

Physical Memory Protection is a RISC-V hardware mechanism that restricts physical memory accesses according to programmable address regions and permission bits. It can control instruction fetches, loads, and stores. It is used to protect trusted code and data, isolate software components, and prevent unintended accesses to sensitive memory.

The code can be examined and cloned via this link:

<https://github.com/rgoksel/Cores-VeeR-EH2-PD-UPC.git>

2 Motivation

The objective of this project is to integrate PMP support into VeeR EH2 processor and validate it using the processor simulation. The selected VeeR EH2 baseline is an M-mode only design with two hardware threads.

The project objectives are:

- integrate PMP configuration and address state into the processor
- support OFF, TOR, NA4, and NAPOT address matching modes
- enforce read, write, and execute permissions
- generate instruction, load, and store access faults
- maintain independent PMP configuration for both hardware threads

create a reproducible processor level regression

3 Methodology and Tools

The project uses an iterative RTL integration and verification workflow. The existing methodologies for PMP checker, PMP CSR logic, instruction fetch path, and load store path are studied. PMP configuration state is traced from the TLU/CSR subsystem to the instruction fetch unit and load store unit. Directed RISC-V assembly programs, with AI assistance, are created and executed on the complete Verilator CPU model.

The main tools used are:

SystemVerilog RTL from the VeeR EH2 source

Verilator for processor simulation

RISC-V GNU toolchain for assembly compilation

execution traces and GTKWave for debugging

The verification approach uses reset isolated firmware binaries. This is necessary because PMP entries with the lock bit set cannot be modified until reset. Keeping the scenarios independent prevents lock state created by one test from influencing later tests.

4 Module Description

4.1 PMP Checker

The RISC-V Privileged Architecture specification is used as the reference for the implementation [1]. The Physical Memory Protection module is based on the model defined in Section 3.7 [1]. This section defines the PMP CSR representation, configuration field encoding, address matching modes, locking rules, privilege behavior, priority rule, and access fault behavior.

The PMP decision logic is implemented in the eh2_pmp module. It receives PMP configuration bytes, PMP address registers, access address, access size, access type, and privilege mode. It produces an allow or deny decision and the associated access fault cause.

Each PMP entry contains one 8 bit configuration field and one address register. The configuration field contains the L bit in bit 7, the A field in bits 4 and 3, and the X, W, and R permission bits in bits 2, 1, and 0. For RV32, the PMP address register encodes physical address bits [33:2].

The design uses the lowest numbered 16 PMP entries rather than the 64 entry option. This keeps the implementation and verification scope manageable. The design and regression target

the $G = 0$ configuration, which provides 4 byte PMP granularity and allows NA4 regions. The implemented address matching modes follow the specification [1]. Table 1 summarizes them, and Table 2 gives the NAPOT size encoding:

OFF: the entry is disabled and does not match

TOR: top of range matching, using the previous PMP address as the lower bound and the current PMP address as the upper bound. For entry 0, the lower bound is zero

NA4: a naturally aligned four byte region

NAPOT: a naturally aligned power of two region, where the low order bits of the address register encode the region size

A	Name	Description
0	OFF	Disabled, matches no address
1	TOR	Top of range
2	NA4	Naturally aligned four byte region
3	NAPOT	Naturally aligned power of two region

Table 1: Encoding of the A field in the PMP configuration register [1].

Low order pmpaddr bits	Mode	Region size
yyyy...yyy	NA4	4 bytes
yyyy...yyy0	NAPOT	8 bytes
yyyy...yy01	NAPOT	16 bytes
yyyy...y011	NAPOT	32 bytes
...	NAPOT	...

Table 2: NAPOT region size encoded in the low order address bits [1].

For each entry the checker computes two conditions: whether the entry overlaps any byte of the access and whether it covers every byte. The entry covers the access only when it matches all bytes from the start address to the final byte. An access that begins inside a region but extends past its end does not match that region. The processor level regression uses aligned accesses that are either entirely inside or entirely outside the configured region. The standalone unit testbench also checks multibyte accesses that span a region boundary and the partial overlap priority case described in the specification.

4.2 Permissions, Priority, and Faults

Instruction fetches require execute permission, loads require read permission, and stores require write permission. When more than one PMP entry overlaps an access, the lowest numbered

entry that matches any byte determines the result. That entry must also cover every byte of the access. Otherwise the access faults regardless of its lock and permission bits.

A denied access results in the fault categories defined in the specification:

instruction access fault for a fetch without execute permission

load access fault for a load without read permission

store or AMO access fault for a store, store conditional, or AMO without write permission

The R, W, and X bits form a collective WARL field in the specification. The reserved combination with $R = 0$ and $W = 1$ must not grant access, so the access checker treats this combination as denied. The PMP checker manages the stored configuration state. CSR write and readback behavior are handled by the surrounding CSR and TLU control logic.

4.3 Privilege Mode Scope

The eh2_pmp module accepts M-mode, S-mode, and U-mode privilege values at its interface. For matching S-mode and U-mode accesses, the R, W, and X permission bits determine whether the access succeeds. If at least one PMP entry is implemented but no entry matches an S-mode or U-mode access, the module denies the access.

For M-mode accesses, the L bit determines whether permissions are enforced. When L is clear, an M-mode access matching an entry succeeds without being restricted by the R, W, and X bits. When L is set, the permissions are enforced for M-mode as well. An M-mode access that matches no PMP entry succeeds. The basic bypass test and the locked entry tests verify these behaviors.

The VeeR EH2 baseline in this project is architecturally M-mode only. All production PMP instances in the instruction fetch and load store paths therefore evaluate accesses in M-mode. The standalone PMP unit testbench can drive S-mode and U-mode privilege values directly. The processor level regression does not run S-mode or U-mode code. That would require current privilege state, stored mstatus.MPP state, and privilege transitions through traps and mret.

5 Integration Description

5.1 PMP Configuration State

PMP configuration state is managed through the processor CSR and TLU subsystem using pmpcfg and pmpaddr registers. The implementation supports 16 PMP entries per hardware thread. The main integration requirement is per thread isolation. PMP configuration written

by one hart must not affect the other. During integration, the per thread PMP state must reach the access check paths separately for each hart. Otherwise an access on one hart can be checked against the other hart configuration. The design delivers a separate PMP bank to each hart, selected by the thread identifier of the transaction.

5.2 Instruction Fetch and Load/Store Integration

PMP enforcement is integrated into the following processor paths:

instruction fetch path for execute permission checks

load path for read permission checks

store path for write permission checks

exception path for propagation of PMP access faults

The flow is shown in Figure 1. Per thread PMP state from the TLU feeds the two PMP banks. The instruction fetch and load store access checkers use these banks. A denied access is raised on the exception path.

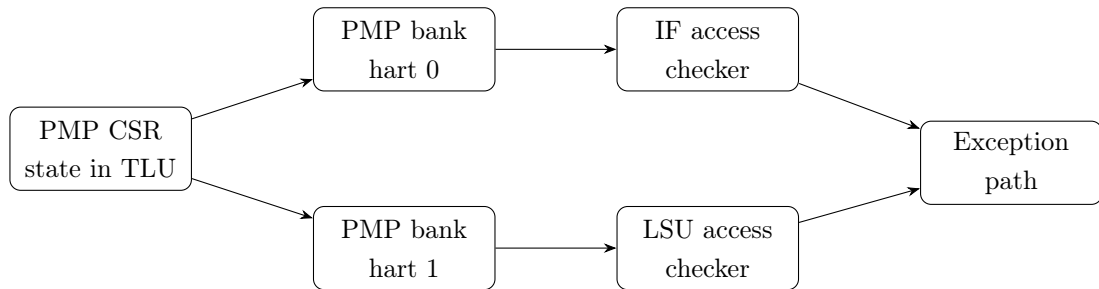


Figure 1: PMP integration flow from per thread state in the TLU to the access checkers and the exception path.

6 Tests and Results

6.1 Test Strategy

Verification uses reset isolated RISC-V assembly programs on the full VeeR EH2 Verilator model. The RISC-V GNU toolchain assembles and links each program into a firmware binary. The Verilator model then executes it. Each program configures PMP state, performs a target access, and checks whether the result or trap cause matches the expected behavior.

Tests that contain locked PMP entries remain in separate binaries since lock state persists until reset. This makes every test independent, reproducible, and easier to debug.

6.2 Directed Regression Tests

Test	Coverage	Checks
Basic CSR and bypass	PMP CSR access and unlocked M-mode bypass	3
Locked TOR	Locked TOR allowed load and denied store	2
Fetch deny	Locked execute deny instruction fetch fault	1
NAPOT, both harts	NAPOT permission enforcement on both hardware threads	4
A equals OFF	Inactive locked A equals OFF behavior	1
Hart 1 read deny	Hart 1 load access fault	1
Hart 1 TOR	Hart 1 TOR read and write access	1
Hart 1 NA4	Hart 1 NA4 read and write access	1
Priority	Lowest numbered matching entry controls the result	2
Lock and WARL	Locking, WARL behavior, persisted lock state, and read denial	5
Total		21

Table 3: Reset isolated PMP regression tests.

The processor level regression executes M-mode software, the only architectural execution mode available in the VeeR EH2 baseline. The standalone PMP unit testbench separately verifies S-mode and U-mode permission decisions by driving the privilege mode input directly. The standalone test can be found in the `pmp/tb` directory and is run with:

```
cd pmp/test_pmp
make run
```

The standalone testbench drives the checker directly with the access address, size, type, and privilege mode. It adds the M-mode and U-mode cases; groups for S-mode permission decisions, multibyte accesses that span a region boundary, the empty TOR range, and the partial overlap priority case. The full run reports:

```
PMP checker results: 84 / 84 passed
ALL TESTS PASSED
```

The S-mode group confirms that S-mode accesses obey the R, W, and X bits and that the lock bit does not change S-mode behavior. The multibyte and partial overlap groups confirm that an access faults unless the deciding entry covers every byte.

6.3 Reproducible Regression

The complete regression is executed from the tools directory:

```
cd tools
make pmp-regression
```

The Make target simulates each firmware test independently. It stores per test simulation logs, console logs, and execution traces under the tools/logs/pmp directory. The regression fails if a test reports failure or does not report a passing result.

The final results are:

```
PASS  pmp_test_new_basic
PASS  pmp_test_new_ltor
PASS  pmp_test_new_fetch
PASS  pmp_test_new_napot
PASS  pmp_test_new_aoff
PASS  pmp_test_new_h1_read_deny
PASS  pmp_test_new_h1_tor
PASS  pmp_test_new_h1_na4
PASS  pmp_test_new_priority
PASS  pmp_test_new_lock_warl
PMP regression: 10/10 passed
```

6.4 Results Discussion

The regression validates PMP behavior for the tested instruction fetch, load, and store paths. The hart 1 tests show that PMP configuration on one hardware thread does not affect the other, so the two threads are isolated.

The priority test confirms the rule that the lowest numbered matching PMP entry controls the final permission decision. The lock and WARL test confirms the behavior that locked PMP configuration and address values cannot be overwritten and remain active after an attempted clear operation.

7 Conclusions

This project implements and validates the PMP module based on M-mode instruction fetch, load, and store protection in the VeeR EH2 baseline.

The final validation contains 10 reset isolated firmware binaries and 21 directed checks. The reproducible regression result is:

PMP regression: 10/10 passed

The implemented scope is M-mode PMP enforcement in the VeeR EH2 baseline, which only supports M-mode. Full system level U-mode enforcement would require a broader processor privilege mode implementation. The partial overlap multibyte priority case is verified at the checker unit level rather than in the processor regression.

References

- [1] RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Chapter 3.7, Physical Memory Protection.
- [2] CHIPS Alliance. VeeR EH2 RISC-V Core Repository.
- [3] Verilator Project. Verilator Documentation.
- [4] Processor Design course materials and project statement.